EXPERIMENT 8: TF-IDF, Cosine Similarity, and PMI

AIM:

To implement Term Frequency-Inverse Document Frequency (TF-IDF) vectorization, compute Cosine Similarity between documents, and calculate Pointwise Mutual Information (PMI) for word pairs.

PROCEDURE:

1. Import numpy and re, Counter from collections.
2. Define sample documents.
3. Preprocess each document (lowercase, remove non-alpha, split).
4. Build vocabulary from all documents.
5. TF-IDF: Compute tf(word, doc) = count/doc_length and idf(word) = log(N/(1+df)).
6. For each document, compute TF-IDF vector over the vocabulary.
7. Cosine Similarity: Define cosine(v1, v2) = dot(v1,v2)/(norm(v1)*norm(v2)).
8. PMI: Compute p(w) = count(w)/total words.
9. Compute p_pair(w1, w2) = co-occurrence count / total words.
10. PMI(w1, w2) = log2(p_pair / (p(w1)*p(w2))).
11. Print TF-IDF vector for Doc1, Cosine Similarity, and PMI for ('language', 'models').

CODE:

```
import numpy as np import re
from collections import Counter
docs = [
    "language models learn patterns",
    "language processing helps computers",
    "models understand language" ]
def preprocess(text):
    text = text.lower() text =
    re.sub(r'[^a-z\s]', '', text) return
    text.split()
docs_tokens = [preprocess(doc) for doc in docs] vocab =
list(set(word for doc in docs_tokens for word in doc))
def tf(word, doc):
    return doc.count(word)/len(doc)
def idf(word):
    return np.log(len(docs)/(1 + sum(word in doc for doc in docs_tokens)))
```

```
tfidf = []
for doc in docs_tokens:
    vec = []
    for word in vocab:
        vec.append(tf(word, doc)*idf(word))
    tfidf.append(vec)

print("\nTF-IDF Vector (Doc1):")
print(tfidf[0])

def cosine(v1, v2):
    return np.dot(v1,v2)/(np.linalg.norm(v1)*np.linalg.norm(v2))

print("\nCosine Similarity (Doc1 vs Doc2):")
print(cosine(tfidf[0], tfidf[1]))

all_words = [w for doc in docs_tokens for w in doc]
total = len(all_words)
word_counts = Counter(all_words)

def p(w): return word_counts[w]/total
def p_pair(w1, w2):
    count = 0
    for doc in docs_tokens:
        for i in range(len(doc)-1):
            if doc[i]==w1 and doc[i+1]==w2: count+=1
    return count/total

def PMI(w1, w2):
    return  np.log2(p_pair(w1,w2)/(p(w1)*p(w2))+1e-6)+1e-6

print("\nPMI(language, models):")
print(PMI("language","models"))
```

OUTPUT:

```
TF-IDF Vector (Doc1):
[np.float64(0.1013662770270411), np.float64(0.1013662770270411), np.float64(0.0),
np.float64(0.0), ...]


Cosine Similarity (Doc1 vs Doc2):
0.16998386627218248


PMI(language, models): 0.8744408107997369
```

RESULT:

TF-IDF vectorization, Cosine Similarity, and PMI were successfully computed. The low cosine similarity (0.17) between Doc1 and Doc2 indicates they share few common important terms. The positive PMI value for ('language', 'models') confirms these words tend to co-occur more than expected by chance, indicating a meaningful association.